Study Guide: AI-Powered Exercise Repetition Tracking App

Overview

This guide walks you through every layer of knowledge needed to build a fitness app that:

- Records training sessions via camera/sensors
- Identifies exercise types automatically
- Counts repetitions in real time
- Tracks performance over time
- Generates session analytics & next-session recommendations

The guide is structured in **5 Learning Phases**, each with concepts to master, questions to test yourself, and implementation checkpoints.

Phase 1: Foundations — Computer Vision & Pose Estimation

1.1 Core Concepts to Learn

Concept	Why It Matters
Human Pose Estimation	Detecting body keypoints (joints) from video frames
Skeleton / Keypoint Models	How 17–33 landmarks map the human body
Optical Flow	Tracking motion between consecutive frames
Depth Estimation	Understanding 3D movement from 2D input
MediaPipe Pose / BlazePose	Google's real-time pose detection library
OpenPose	Multi-person pose estimation research baseline
MoveNet (TensorFlow Lite)	Lightweight on-device pose model

1.2 Key Questions to Answer

1. What are keypoints and how does a model output them?
2. What is the confidence score of a keypoint and why does it matter?
3. How does joint angle calculation work from (x, y) coordinates?
4. What is the difference between 2D and 3D pose estimation?

1.3 Hands-On Exercises

- [ ] Run MediaPipe Pose on a webcam feed and visualize the skeleton overlay
- [ ] Print out all 33 landmark coordinates per frame
- [ ] Calculate the elbow angle using shoulder, elbow, and wrist keypoints
- [ ] Record a video of a bicep curl and plot the elbow angle over time

1.4 Resources

- MediaPipe Pose Docs
- MoveNet TF Hub
- Paper: *BlazePose: On-device Real-time Body Pose tracking* (Bazarevsky et al., 2020)

Phase 2: Exercise Recognition & Rep Counting

2.1 Core Concepts to Learn

Concept	Why It Matters
Joint Angle Time Series	The primary signal for detecting exercise phases
State Machine (Up/Down)	Simple, robust rep counter logic
Peak Detection	Finding maxima/minima in angle curves
Sliding Window Classification	Classify exercise type from recent N frames
Feature Engineering	Deriving velocity, acceleration from raw keypoints
Sequence Models (LSTM, TCN)	Classify exercise type from temporal sequences

DTW (Dynamic Time Warping)	Template matching for gesture/movement recognition
Threshold Tuning	Avoiding false positives in rep counting

2.2 Rep Counting Logic — State Machine Model

```
State: NEUTRAL → DOWN (angle decreases past threshold) → UP (angle returns past threshold) → count +1
```

Implement this with:

1. A joint angle signal (e.g., knee angle for squats)
2. Two thresholds: `down_threshold` and `up_threshold`
3. A state variable: `"up"` or `"down"`
4. A counter that increments on each full cycle

2.3 Exercise Classification Strategy

Rule-Based (beginner)

- Map exercises to primary joint(s): squat → knee, bicep curl → elbow, shoulder press → elbow + shoulder
- Detect exercise by which joint angle oscillates most

ML-Based (advanced)

- Collect labeled video clips per exercise
- Extract keypoint sequences as feature vectors
- Train an LSTM or 1D-CNN to classify exercise type
- Inference on rolling 2–5 second windows

2.4 Key Questions to Answer

1. Why use a two-threshold state machine instead of a single threshold?
2. How would you handle occlusion (a keypoint going off-screen)?
3. What features distinguish a squat from a lunge?
4. How does DTW handle speed variation between reps?

2.5 Hands-On Exercises

- [ ] Build a rep counter for bicep curls using the elbow angle
- [ ] Add a second exercise (squats) using the knee angle
- [ ] Implement a simple rule-based exercise classifier for 3 exercises
- [ ] Visualize a rep-by-rep angle curve for an entire set

Phase 3: Data Modeling & Performance Tracking

3.1 Core Concepts to Learn

Concept	Why It Matters
Time-Series Data Storage	Persisting per-rep, per-set, per-session data
Relational Schema Design	Structuring sessions, sets, exercises relationally
Session Normalization	Comparing performance across different session lengths
Progressive Overload Metrics	Volume (sets × reps × weight), 1RM estimation
Fatigue Indicators	Rep velocity decline, range-of-motion reduction
REST API Design	Exposing session data to frontend

3.2 Recommended Data Schema

```
User
 └─── Session (date, duration, notes)
       └─── Exercise (name, type)
             └─── Set (set_number, reps_counted, avg_rep_duration, avg_rom_angle)
                   └─── Rep (rep_number, peak_angle, valley_angle, duration_ms, velocity_score)
```

3.3 Performance Metrics to Track Per Session

Metric	Formula / Description

Total Volume	Sum of (reps × weight) across all sets
Avg Rep Duration	Mean time between state transitions
Range of Motion (ROM)	Peak angle − Valley angle per rep
Velocity Score	ROM / duration per rep (proxy for power)
Fatigue Index	Velocity in last 3 reps / velocity in first 3 reps
Consistency Score	Standard deviation of rep durations

3.4 Key Questions to Answer

1. Why is ROM tracking important beyond just counting reps?
2. How would you store rep-level data efficiently at scale?
3. What is the 1RM (One Rep Max) formula and how is it estimated from performance data?
4. How can you detect compensation patterns (e.g., leaning too far forward in a squat)?

3.5 Hands-On Exercises

- [ ] Design a SQLite or Firebase schema for the data model above
- [ ] Store a session's rep data after a live tracking session
- [ ] Query average ROM across all sessions for one exercise
- [ ] Calculate the Fatigue Index for a recorded set

Phase 4: Analytics Dashboard & Visualization

4.1 Core Concepts to Learn

Concept	Why It Matters
Time-Series Charting	Visualizing progress over weeks/months
Heatmaps	Showing muscle activation or workout frequency
Radar Charts	Comparing performance across multiple metrics

Trend Analysis	Moving averages, week-over-week change
Data Normalization for Display	Making metrics comparable on the same chart

4.2 Analytics Features to Implement

Session Summary Screen

- Total reps, sets, volume
- Rep-by-rep angle curve (line chart)
- Rep duration over the set (bar chart — detects fatigue)
- ROM distribution histogram

Progress Tracking Screen

- Volume over time (line chart per exercise)
- Personal records (PRs) highlighted
- Streak / consistency calendar

Form Analysis Screen

- Average keypoint position heatmap
- Comparison of best rep vs. worst rep (overlay)

4.3 Recommended Libraries

Layer	Library
React frontend charts	Recharts, Chart.js, Victory
Mobile charts	Victory Native, react-native-svg-charts
Python data analysis	Pandas, NumPy, SciPy
Python visualization	Matplotlib, Plotly

4.4 Hands-On Exercises

- [ ] Build a bar chart showing rep duration for each rep in a set
- [ ] Plot ROM over multiple sessions for one exercise
- [ ] Create a weekly volume line chart with a 4-week moving average
- [ ] Display a "best rep vs. worst rep" angle curve overlay

Phase 5: Recommendation Engine

5.1 Core Concepts to Learn

Concept	Why It Matters
Progressive Overload Principle	Foundation of all strength training recommendations
Rule-Based Recommendation Systems	Simple, transparent, interpretable logic
Collaborative Filtering (advanced)	Recommend based on similar users' data
LLM-Powered Recommendations	Use Claude/GPT to generate natural language insights
Readiness / Recovery Modeling	Incorporate rest days, session intensity

5.2 Recommendation Logic — Rule-Based Starter

```
IF avg_velocity_score IMPROVED > 10% AND consistency_score > 0.85:
  → "Great session! Try adding 1 more set next time."

IF fatigue_index < 0.70:
  → "Signs of fatigue detected. Consider reducing sets by 1 or adding rest time."

IF avg_ROM < baseline_ROM * 0.90:
  → "Range of motion was lower than usual. Focus on full extension next session."

IF 3 consecutive sessions show volume plateau:
  → "Plateau detected. Consider varying rep range (e.g., 3×12 → 5×5)."
```

5.3 LLM Integration for Natural Language Recommendations

You can call the Anthropic API with session analytics data and prompt Claude to generate a personalized recommendation paragraph:

```javascript
const prompt = `
You are a personal trainer AI.
Here is the user's session data: ${JSON.stringify(sessionSummary)}
Their recent history: ${JSON.stringify(recentSessions)}
Generate a short, encouraging, specific recommendation for their next session.
Focus on: volume adjustment, form cues, rest time, and any fatigue signals.
`;
```

5.4 Key Questions to Answer

1. What is the difference between a rule-based and ML-based recommender?
2. Why is fatigue detection important for preventing injury?
3. How would you avoid over-fitting recommendations to a single noisy session?
4. How do you present recommendations to users without overwhelming them?

5.5 Hands-On Exercises

- [ ] Implement 3 rule-based recommendation triggers
- [ ] Integrate Claude API to generate a plain-English recommendation paragraph
- [ ] Display next-session recommendation on a summary screen
- [ ] A/B test two recommendation phrasings and log which one users act on

---

Full App Architecture Summary

```
|          Mobile / Web App          |
|                                                        |
|  | Camera  |   | Dashboard |   | Recs    |  |
|  | + Live  |   | Analytics |   | Screen  |  |
|  | Counter |   |           |   |         |  |
|                                                        |
```

```
│            Backend API             │
│ Pose Engine  │ Session DB  │ Rec Engine  │
│ (MediaPipe / │ (SQLite /   │ (Rules +    │
│  MoveNet)    │  Firebase)  │  Claude AI) │
```

Technology Stack Options

Component	Beginner Stack	Advanced Stack
Pose Estimation	MediaPipe (Python)	MoveNet + custom LSTM
Backend	FastAPI + SQLite	Node.js + PostgreSQL
Frontend	React + Recharts	React Native (mobile)
Recommendations	Rule-based Python	Claude API
Deployment	Local / Ngrok	AWS / GCP + Docker

Suggested Learning Sequence (8 Weeks)

Week	Focus	Deliverable
1	Phase 1: Pose estimation basics	Live skeleton overlay on webcam
2	Phase 2: Joint angles + state machine	Working bicep curl counter
3	Phase 2: Exercise classifier	Classify 3 exercises in real time
4	Phase 3: Data schema + storage	Session saved to database
5	Phase 4: Analytics dashboard	Session summary chart screen
6	Phase 4: Progress tracking	Multi-session trend charts
7	Phase 5: Recommendations	Rule-based + Claude API recs
8	Integration + polish	Full end-to-end demo

Self-Assessment Checklist

Before considering yourself ready, ensure you can:

- [ ] Explain how pose estimation extracts joint keypoints from a video frame
- [ ] Implement a rep counter from scratch using a state machine
- [ ] Design a relational schema for sessions, sets, and reps
- [ ] Build at least 3 analytics charts from session data
- [ ] Write and justify 5 recommendation rules based on training science
- [ ] Integrate an LLM API to generate natural-language coaching feedback
- [ ] Connect all layers in a single working demo

This guide was designed to develop both conceptual understanding and practical implementation skills. Work through each phase in order, completing the hands-on exercises before advancing.